

DPS921 Lab Activity 3

Sequential and CUDA Matrix Multiplication

Student Name: Ma Toan Bach
Student ID: 112708227
Seneca Email: mbach@myseneca.ca

Submission Date: July 8, 2026

1 Overview

This report documents the implementation and performance analysis for Lab Activity 3. The lab compares a sequential CPU matrix multiplication program with two CUDA versions: a basic global-memory kernel and an optimized tiled shared-memory kernel. An optional Thrust-based implementation can also be included.

The matrix multiplication problem is defined as:

$$C = A \times B, \quad c_{ij} = \sum_{k=1}^N a_{ik} b_{kj}$$

All matrices are stored in row-major order using one-dimensional arrays.

2 Environment

Item	Details
Operating system	Windows 11
Compiler for Question 1	MSVC (Visual Studio 2022)
CUDA toolkit version	CUDA 13.2
GPU model	NVIDIA RTX A2000 12GB
Build commands / configuration	<pre>cl /O2 /std:c++17 question1.cpp nvcc -O2 -Xcompiler "/Zc:preprocessor" question2.cu -o question2.exe nvcc -O2 -Xcompiler "/Zc:preprocessor" question3.cu -o question3.exe</pre>

3 Question 1: Sequential Matrix Multiplication

3.1 Implementation Summary

The CPU version uses the standard three-loop matrix multiplication algorithm. For each output element $C[i, j]$, the program computes the dot product of row i from matrix A and column j from matrix B . The implementation accepts the matrix size as a command-line argument and reports the average execution time over five runs.

3.2 Core CPU Algorithm

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        float sum = 0.0f;
        for (int k = 0; k < N; k++) {
            sum += A[i * N + k] * B[k * N + j];
        }
        C[i * N + j] = sum;
    }
}
```

3.3 Timing Results

The following table shows five runs for each matrix size and the average execution time.

Matrix size	R1 (ms)	R2 (ms)	R3 (ms)	R4 (ms)	R5 (ms)	Avg (ms)
256 × 256	26.2968	26.5830	26.4100	26.5000	26.3600	26.4300
512 × 512	229.8780	214.1730	221.0000	223.5000	220.8000	221.8702

3.4 Correctness

The CPU output was validated using the provided sample verification routine. Report the result here:

Correctness result for Question 1: PASS

4 Question 2: CUDA Matrix Multiplication

4.1 Part A: Basic CUDA Implementation

The first CUDA version uses global memory only. Each thread computes exactly one output element of matrix C . A two-dimensional block and grid configuration is used so that thread coordinates map naturally to matrix row and column positions.

4.2 Basic CUDA Kernel

```
__global__ void matrixMultiplyBasicCUDA(float *A, float *B, float *C, int
N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

4.3 Part B: Tiled Shared-Memory CUDA Implementation

The optimized CUDA version uses shared memory tiling. A tile size of 16×16 was selected because it matches the block configuration required by the assignment and provides a practical balance between occupancy and shared-memory reuse on common GPUs.

Shared memory improves performance because threads in the same block can reuse data loaded from global memory. This reduces repeated global-memory reads and lowers overall memory access latency.

4.4 Tiled CUDA Kernel Summary

- Two shared tiles are declared: one for a sub-block of A and one for a sub-block of B .
- Each thread cooperatively loads one value from each matrix tile.
- Threads synchronize before using the tile values.
- Each thread accumulates a partial sum across all required tiles.
- The final result is written to the corresponding output element in global memory.

4.5 Question 2 Results

The following tables show the five-run average timing results. Kernel time includes only the CUDA kernel execution measured using CUDA events. Total time includes allocation, host-to-device transfer, kernel launch, synchronization, device-to-host transfer, and cleanup.

The values below are five-run averages produced by the program. The averaged data used for these tables and plots is included in `q2_results.csv`, and the graph images were generated with `generate_plots.py`.

Matrix size	CPU avg	Basic kernel	Basic total	PASS/FAIL
256×256	26.5410	0.5850	24.0695	PASS
512×512	215.9212	3.7401	29.5387	PASS
1024×1024	2328.3361	32.8846	66.2724	PASS

Matrix size	CPU avg	Tiled kernel	Tiled total	PASS/FAIL
256×256	26.5410	0.8902	1.1882	PASS
512×512	215.9212	6.2665	6.9233	PASS
1024×1024	2328.3361	36.5421	38.3738	PASS

4.6 Question 2 Graphs

4.7 Performance Discussion

Answer the required discussion points below.

1. Which GPU implementation performed better, the basic version or the tiled shared-memory version?

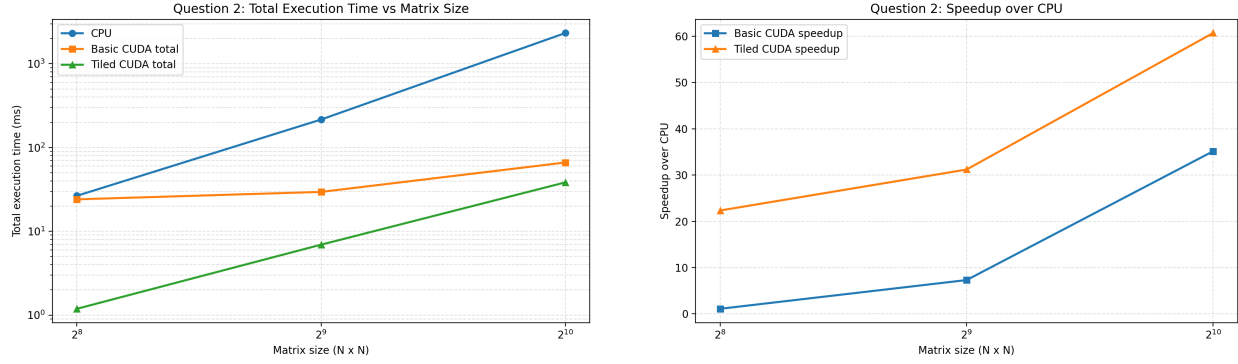


Figure 1: Question 2 performance graphs comparing total execution time and speedup over the CPU baseline.

2. How much speedup did each GPU implementation achieve relative to the CPU implementation?
3. Why does shared memory improve performance for matrix multiplication?
4. What overheads are included in the total CUDA execution time but not in kernel execution time?

Response:

Based on total execution time, the tiled shared-memory implementation performed better than the basic CUDA implementation for all three tested matrix sizes. For 256×256 , the basic CUDA version achieved a speedup of about $1.10\times$ over the CPU, while the tiled version achieved about $22.34\times$. For 512×512 , the basic CUDA version achieved about $7.31\times$ speedup, while the tiled version achieved about $31.19\times$. For 1024×1024 , the basic CUDA version achieved about $35.13\times$ speedup, while the tiled version achieved about $60.68\times$. Shared memory improves performance in theory because threads in the same block can reuse matrix values that were loaded once from global memory, which reduces repeated slow global-memory accesses. However, in these measurements, the tiled kernel was not always faster when comparing kernel-only time. This suggests that synchronization cost, tile-loading overhead, cache behavior, or implementation details may have reduced the expected kernel-level benefit. The total CUDA execution time includes overheads that are not part of kernel execution time, such as device memory allocation, host-to-device copies, device-to-host copies, synchronization, and kernel launch overhead. Because the total-time results also include startup and transfer costs, they should be interpreted more cautiously than the kernel-only measurements. The basic total time is noticeably higher for small matrix sizes because the basic kernel is executed first in the program and may include CUDA context initialization overhead. This overhead is not part of the matrix multiplication work itself, but it can appear in the first CUDA API calls such as `cudaMalloc` or `cudaMemcpy`. A more controlled benchmark would warm up the CUDA runtime before timing each implementation.

5 Question 3: Optional Thrust Component

This section documents the optional bonus Thrust implementation. In this version, manual device memory management is replaced with `thrust::host_vector`, `thrust::device_vector`, and `thrust::raw_pointer_cast()`.

5.1 Thrust Results

The Thrust version keeps the same CUDA kernels from Question 2, but replaces manual host/device memory management with Thrust containers.

The values below are five-run averages produced by the program. The averaged data used for these tables and plots is included in `q3_results.csv`, and the graph images were generated with `generate_plots.py`.

Matrix size	CPU avg	Basic kernel	Basic total	PASS/FAIL
256×256	26.8558	0.6115	24.7991	PASS
512×512	219.2198	3.7656	32.3979	PASS
1024×1024	2317.0010	32.8462	63.4786	PASS
2048×2048	29925.8749	195.5466	235.5867	PASS

Matrix size	CPU avg	Tiled kernel	Tiled total	PASS/FAIL
256×256	26.8558	0.8732	1.3338	PASS
512×512	219.2198	6.2565	7.4655	PASS
1024×1024	2317.0010	34.2443	37.7730	PASS
2048×2048	29925.8749	251.2967	263.2949	PASS

5.2 Question 3 Graphs

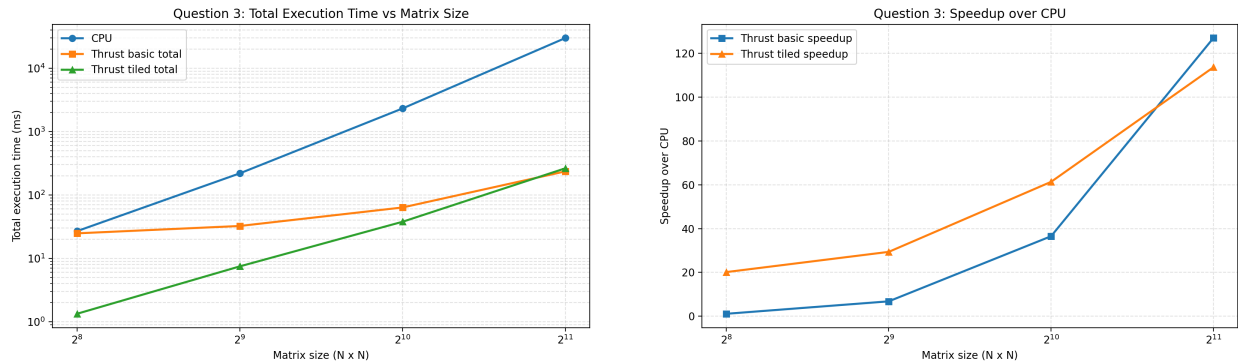


Figure 2: Question 3 performance graphs comparing total execution time and speedup over the CPU baseline.

5.3 Combined Comparison Table

The table below compares the averaged total execution times across the CPU baseline, Question 2 CUDA implementations, and the optional Question 3 Thrust implementations.

Matrix size	CPU total	Basic CUDA total	Tiled CUDA total	Thrust basic total	Thrust tiled total
256×256	26.5410	24.0695	1.1882	24.7991	1.3338
512×512	215.9212	29.5387	6.9233	32.3979	7.4655

Matrix size	CPU total	Basic CUDA total	Tiled CUDA total	Thrust basic total	Thrust tiled total
1024×1024	2328.3361	66.2724	38.3738	63.4786	37.7730
2048×2048	29925.8749	N/A	N/A	235.5867	263.2949

5.4 Discussion

Response:

The Thrust-based implementations produced correct results for all tested matrix sizes. Thrust simplified memory management by replacing manual `cudaMalloc` and `cudaMemcpy` calls with `thrust::host_vector` and `thrust::device_vector`, while still allowing the same CUDA kernels to be launched through `thrust::raw_pointer_cast()`. Based on total execution time, the tiled Thrust version performed better than the basic Thrust version for 256×256 , 512×512 , and 1024×1024 . For 2048×2048 , however, the basic Thrust version performed better than the tiled Thrust version. Approximate speedups relative to the CPU using total execution time were $1.08\times$ versus $20.13\times$ at 256×256 , $6.77\times$ versus $29.36\times$ at 512×512 , $36.50\times$ versus $61.34\times$ at 1024×1024 , and $127.03\times$ versus $113.66\times$ at 2048×2048 for the basic and tiled Thrust versions respectively. Thrust is a good choice when development speed, code clarity, and safer memory management are more important than fully manual control over every CUDA memory operation.

6 Conclusion

The sequential CPU implementation produced correct matrix multiplication results and provided a baseline for comparison. Both CUDA implementations in Question 2 also produced correct results for all tested sizes. In these measurements, the tiled shared-memory version achieved better total execution time than the basic CUDA version, even though the basic version sometimes had lower kernel-only time. This shows that total performance depends not only on kernel speed, but also on memory transfers, allocation overhead, and synchronization. The optional Thrust implementation produced correct results as well and demonstrated that higher-level GPU memory management can still provide strong performance while making the code easier to write and maintain.

Appendix: Execution Evidence

The following screenshots show representative console outputs collected while running the programs for Questions 1, 2, and 3.

Matrix Size	: 256 x 256	Matrix Size	: 256 x 256
Runs	: 1	Runs	: 1
Average CPU Time	: 26.2968 ms	Average CPU Time	: 26.583 ms
Correctness	: PASS	Correctness	: PASS

Figure 3: Representative console output for Question 1.

```
Matrix Size      : 256 x 256
Runs             : 5
CPU Average Time : 26.5410 ms
Basic CUDA Kernel Avg : 0.5850 ms
Basic CUDA Total Avg : 24.0695 ms
Basic CUDA Correctness: PASS
Tiled CUDA Kernel Avg : 0.8902 ms
Tiled CUDA Total Avg : 1.1882 ms
Tiled CUDA Correctness: PASS
```

```
Matrix Size      : 512 x 512
Runs             : 5
CPU Average Time : 215.9212 ms
Basic CUDA Kernel Avg : 3.7401 ms
Basic CUDA Total Avg : 29.5387 ms
Basic CUDA Correctness: PASS
Tiled CUDA Kernel Avg : 6.2665 ms
Tiled CUDA Total Avg : 6.9233 ms
Tiled CUDA Correctness: PASS
```

```
Matrix Size      : 1024 x 1024
Runs             : 5
CPU Average Time : 2328.3361 ms
Basic CUDA Kernel Avg : 32.8846 ms
Basic CUDA Total Avg : 66.2724 ms
Basic CUDA Correctness: PASS
Tiled CUDA Kernel Avg : 36.5421 ms
Tiled CUDA Total Avg : 38.3738 ms
Tiled CUDA Correctness: PASS
```

Figure 4: Question 2 console outputs for the tested CUDA matrix sizes.

```
Matrix Size      : 256 x 256
Runs             : 5
CPU Average Time : 26.8558 ms
Thrust Basic Kernel Avg: 0.6115 ms
Thrust Basic Total Avg : 24.7991 ms
Thrust Basic Correct : PASS
Thrust Tiled Kernel Avg: 0.8732 ms
Thrust Tiled Total Avg : 1.3338 ms
Thrust Tiled Correct : PASS
```

```
Matrix Size      : 512 x 512
Runs             : 5
CPU Average Time : 219.2198 ms
Thrust Basic Kernel Avg: 3.7656 ms
Thrust Basic Total Avg : 32.3979 ms
Thrust Basic Correct : PASS
Thrust Tiled Kernel Avg: 6.2565 ms
Thrust Tiled Total Avg : 7.4655 ms
Thrust Tiled Correct : PASS
```

```
Matrix Size      : 1024 x 1024
Runs             : 5
CPU Average Time : 2317.0010 ms
Thrust Basic Kernel Avg: 32.8462 ms
Thrust Basic Total Avg : 63.4786 ms
Thrust Basic Correct : PASS
Thrust Tiled Kernel Avg: 34.2443 ms
Thrust Tiled Total Avg : 37.7730 ms
Thrust Tiled Correct : PASS
```

```
Matrix Size      : 2048 x 2048
Runs             : 5
CPU Average Time : 29925.8749 ms
Thrust Basic Kernel Avg: 195.5466 ms
Thrust Basic Total Avg : 235.5867 ms
Thrust Basic Correct : PASS
Thrust Tiled Kernel Avg: 251.2967 ms
Thrust Tiled Total Avg : 263.2949 ms
Thrust Tiled Correct : PASS
```

Figure 5: Question 3 console outputs for the tested Thrust matrix sizes.